CLI and ORM: Code-Along (CodeGrade)

Learning Goals

- Implement a CLI for an ORM application

Key Vocab

- **Command Line**: a text-based interface that is built into your computer's operating system. It allows you to access the files and applications on your computer manually or through scripts.
- **Terminal**: the application in Mac OS that allows you to access the command line.
- **Command Shell/Powershell**: the applications in Windows that allow you to access the command line.
- **Command-Line Interface (CLI)**: a text-based interface used to run programs, manage files and interact with objects in memory. As the name suggests, it is run from the command line.
- **Object-Relational Mapping (ORM)**: a programming technique that provides a mapping between an object-oriented data model and a relational database model.
- **Attribute**: variables that belong to an object.
- **Property**: attributes that are controlled by methods.
- **Decorator**: function that takes another function as an argument and returns a new function with added functionality.

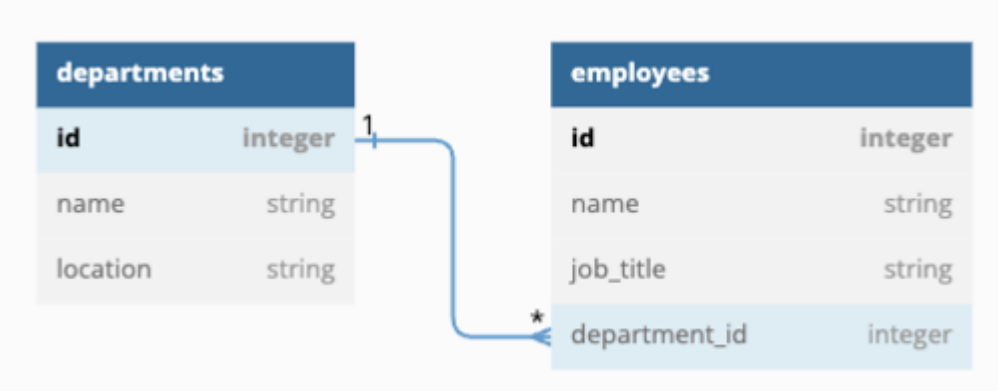
Code Along

Let's implement a CLI to provide a text-based interface to an ORM application. This lesson is a code-along, so fork and clone the repo.

NOTE: Remember to run `pipenv install` to install the dependencies and `pipenv shell` to enter your virtual environment before running your code.

```
pipenv install
pipenv shell
```

We'll add a command line interface to the company ORM application that we've worked with in previous lessons:



Take a look at the directory structure:

```
.
└── lib
    ├── models
        ├── __init__.py
        ├── department.py
    |   └── employee.py
    ├── testing
        ├── conftest.py
        ├── department_orm_test.py
        ├── department_property_test.py
        ├── employee_orm_test.py
    |   └── employee_property_test.py
    ├── cli.py
```

```
    ├── company.db
    ├── debug.py
    ├── helpers.py
    └── seed.py
├── Pipfile
├── Pipfile.lock
├── pytest.ini
├── README.md
```

The `lib/models` folder contains the `Department` and `Employee` class, along with `__init__.py`. There are a few things to note:

- The database environment setup is in `/lib/models/**init**.py`.
- Import statements in the Python files have been evolved to account for the `lib/models` folder.

You should not need to make any changes to `Department` or `Employee`.

Test files

This is **not** a test-driven code-along, although the repo does contain a `lib/testing` folder that tests the current implementation `Department` and `Employee`. If you look over the test files, you can see how to adapt the `import` statements to find classes within the `lib/models` subfolder. This could be useful when you're implementing the Phase 3 project if you choose to use a similar directory structure.

The tests should pass if you run them:

```
pytest -x
```

Seeding the database with sample data

The file `lib/seed.py` contains code to initialize the database with sample departments and employees. Run the following command to seed the database:

```
python lib/seed.py
```

You can use the SQLITE EXPLORER extension to explore the initial database contents. (Another alternative is to run `python lib/debug.py` and use the `ipbd` session to explore the database)

`cli.py` and `helpers.py`

The file `lib/cli.py` contains a command line interface for our company database application. The `main` method has a loop that (1) displays a menu of choices, and then (2) calls a helper function based on the user's choice. The helper functions are contained in `lib/helpers.py` .

If you look at `lib/helpers.py` , you'll notice most of the functions contain the `pass` statement.

Try running `python lib/cli.py` and select a menu choice such as `1` . Since the helper functions contain the `pass` statement, no action is performed on the database. Enter `0` to exit the CLI.

```
Please select an option:
0. Exit the program
1. List all departments
2. Find department by name
3. Find department by id
4: Create department
5: Update department
6: Delete department
7. List all employees
8. Find employee by name
9. Find employee by id
10: Create employee
11: Update employee
12: Delete employee
13: List all employees in a department
> 1
Please select an option:
0. Exit the program
1. List all departments
```

```
2. Find department by name
3. Find department by id
4: Create department
5: Update department
6: Delete department
7. List all employees
8. Find employee by name
9. Find employee by id
10: Create employee
11: Update employee
12: Delete employee
13: List all employees in a department
> 0
Goodbye!
```

We will implement the functions related to the `Department` class in this lesson. You will then implement the functions related to the `Employee` class in the lab.

`list_departments()`

Let's start with the `list_departments()` function in `lib/helpers.py`. The function should get all departments stored in the database, then print each department on a new line. Replace the `pass` statement with the code shown below:

```
def list_departments():
    departments = Department.get_all()
    for department in departments:
        print(department)
```

We can test this new functionality using the CLI. Run `python lib/cli.py`, then enter `1` at the menu prompt to list all departments:

```
Please select an option:
0. Exit the program
```

```
1. List all departments
...
> 1
<Department 1: Payroll, Building A, 5th Floor>
<Department 2: Human Resources, Building C, East Wing>
```

find_department_by_name()

The function `find_department_by_name()` should prompt for a `name`, find the `Department` instance that matches, and print the matching object's data or an error message:

```
def find_department_by_name():
    name = input("Enter the department's name: ")
    department = Department.find_by_name(name)
    print(department) if department else print(
        f'Department {name} not found')
```

Run `python lib/cli.py` to test the function:

```
Please select an option:
0. Exit the program
1. List all departments
2. Find department by name
...
> 2
Enter the department's name: Payroll
<Department 1: Payroll, Building A, 5th Floor>
```

Try entering a name that does not match any department:

```
Please select an option:
0. Exit the program
```

```
1. List all departments
2. Find department by name
...
> 2
Enter the department's name: Sales and Marketing
Department Sales and Marketing not found
```

find_department_by_id()

The function `find_department_by_id()` should prompt for an `id`, find the `Department` instance that matches, and print either the matching object's data or an error message:

```
def find_department_by_id():
    #use a trailing underscore not to override the built-in id function
    id_ = input("Enter the department's id: ")
    department = Department.find_by_id(id_)
    print(department) if department else print(f'Department {id_} not found')
```

Run `python lib/cli.py` to test the function. Test with various id values:

- An id that matches a department instance such as `1` or `2`.
- An id that does not match any departments, i.e. `99`.
- A id value that is not an int, such as `one`.

create_department()

The function `create_department()` should prompt for a name and location, then create and persist a new `Department` class instance. Surround the code in a `try/except` block in case an exception is thrown by the `name` or `location` property setter methods:

```
def create_department():
    name = input("Enter the department's name: ")
    location = input("Enter the department's location: ")
    try:
```

```
            department = Department.create(name, location)
            print(f'Success: {department}')
        except Exception as exc:
            print("Error creating department: ", exc)
```

Let's test the method with valid attribute values, then list all departments to confirm the new department was added:

```
Please select an option:
0. Exit the program
1. List all departments
2. Find department by name
3. Find department by id
4: Create department
5: Update department
6: Delete department
7. List all employees
8. Find employee by name
9. Find employee by id
10: Create employee
11: Update employee
12: Delete employee
13: List all employees in a department
> 4
Enter the department's name: Sales
Enter the department's location: Building B
Success: <Department 3: Sales, Building B>
```

Let's confirm the department was added to the database by listing all departments:

```
Please select an option:
0. Exit the program
1. List all departments
...
```

```
> 1
<Department 1: Payroll, Building A, 5th Floor>
<Department 2: Human Resources, Building C, East Wing>
<Department 3: Sales, Building B>
```

Try entering invalid data for name and location:

```
Please select an option:
...
> 4
Enter the department's name:
Enter the department's location:
Error creating department:  Name cannot be empty and must be a string
```

update_department()

The function `update_department()` should prompt for the department `id`, `name`, and `location`. The function must update the Python object's state as well as update the database row for that object. The function should print an error message if the `id` does not match a row in the table, or if the provided `name` or `location` are not valid.

```
def update_department():
    id_ = input("Enter the department's id: ")
    if department := Department.find_by_id(id_):
        try:
            name = input("Enter the department's new name: ")
            department.name = name
            location = input("Enter the department's new location: ")
            department.location = location

            department.update()
            print(f'Success: {department}')
        except Exception as exc:
```

```
            print("Error updating department: ", exc)
    else:
        print(f'Department {id_} not found')
```

Test the function with valid values for `id`, `name`, and `location`.

```
Please select an option:
...
> 5
Enter the department's id: 1
Enter the department's new name: Payroll and Accounting
Enter the department's new location: Building Z
Success: <Department 1: Payroll and Accounting, Building Z>
```

Confirm the database was updated by listing all departments:

```
Please select an option:
...
> 1
<Department 1: Payroll and Accounting, Building Z>
<Department 2: Human Resources, Building C, East Wing>
<Department 3: Sales, Building B>
```

You should also test by providing an invalid id such as `99`, as well as empty strings for the `name` and `location` to ensure the function prints appropriate error messages.

`delete_department()`

The function `delete_department()` should prompt for the department `id` and delete the department from the database if it exists and print a confirmation message, or print an error message if the department does not exist as shown below:

```
def delete_department():
    id_ = input("Enter the department's id: ")
    if department := Department.find_by_id(id_):
        department.delete()
        print(f'Department {id_} deleted')
    else:
        print(f'Department {id_} not found')
```

Run `python lib/cli.py` and test the delete option with an existing department id such as `1` as well as a non-existent one like `99`.

Conclusion

The CLI front end for an ORM application prompts the user for an action, then calls ORM methods within a helper function to perform the necessary action.

You'll implement the CLI front end for testing the ORM methods of the `Employee` class as part of the next lab.

Solution Code

```
from models.department import Department
from models.employee import Employee


def exit_program():
    print("Goodbye!")
    exit()

# We'll implement the department functions in this lesson

def list_departments():
    departments = Department.get_all()
    for department in departments:
```

```
            print(department)


    def find_department_by_name():
        name = input("Enter the department's name: ")
        department = Department.find_by_name(name)
        print(department) if department else print(
            f'Department {name} not found')


    def find_department_by_id():
        #use a trailing underscore not to override the built-in id function
        id_ = input("Enter the department's id: ")
        department = Department.find_by_id(id_)
        print(department) if department else print(f'Department {id_} not found')


    def create_department():
        name = input("Enter the department's name: ")
        location = input("Enter the department's location: ")
        try:
            department = Department.create(name, location)
            print(f'Success: {department}')
        except Exception as exc:
            print("Error creating department: ", exc)


    def update_department():
        id_ = input("Enter the department's id: ")
        if department := Department.find_by_id(id_):
            try:
                name = input("Enter the department's new name: ")
                department.name = name
```

```
            location = input("Enter the department's new location: ")
            department.location = location

            department.update()
            print(f'Success: {department}')
        except Exception as exc:
            print("Error updating department: ", exc)
    else:
        print(f'Department {id_} not found')


def delete_department():
    id_ = input("Enter the department's id: ")
    if department := Department.find_by_id(id_):
        department.delete()
        print(f'Department {id_} deleted')
    else:
        print(f'Department {id_} not found')


# You'll implement the employee functions in the lab

def list_employees():
    pass


def find_employee_by_name():
    pass


def find_employee_by_id():
    pass
```

```
    def create_employee():
        pass


    def update_employee():
        pass


    def delete_employee():
        pass


    def list_department_employees():
        pass
```